On-call Runbook Copilot

An on-call assistant that retrieves the matching runbook, proves the root cause from live logs, and cites its evidence. Evaluated on 55 faults injected into a real API.

Ashutosh · Technical documentation · October 2026

Live demo: ashu-stark.github.io/oncall-copilot/ · GitHub: github.com/Ashu-Stark/oncall-copilot

Contents

1. Overview

2. The problem

3. Architecture

4. How an incident is investigated

5. Retrieval in detail

6. The runbooks

7. Fault injection

8. Evaluation

9. LLM investigator (optional)

10. Safety and design decisions

11. Limitations

12. Running it

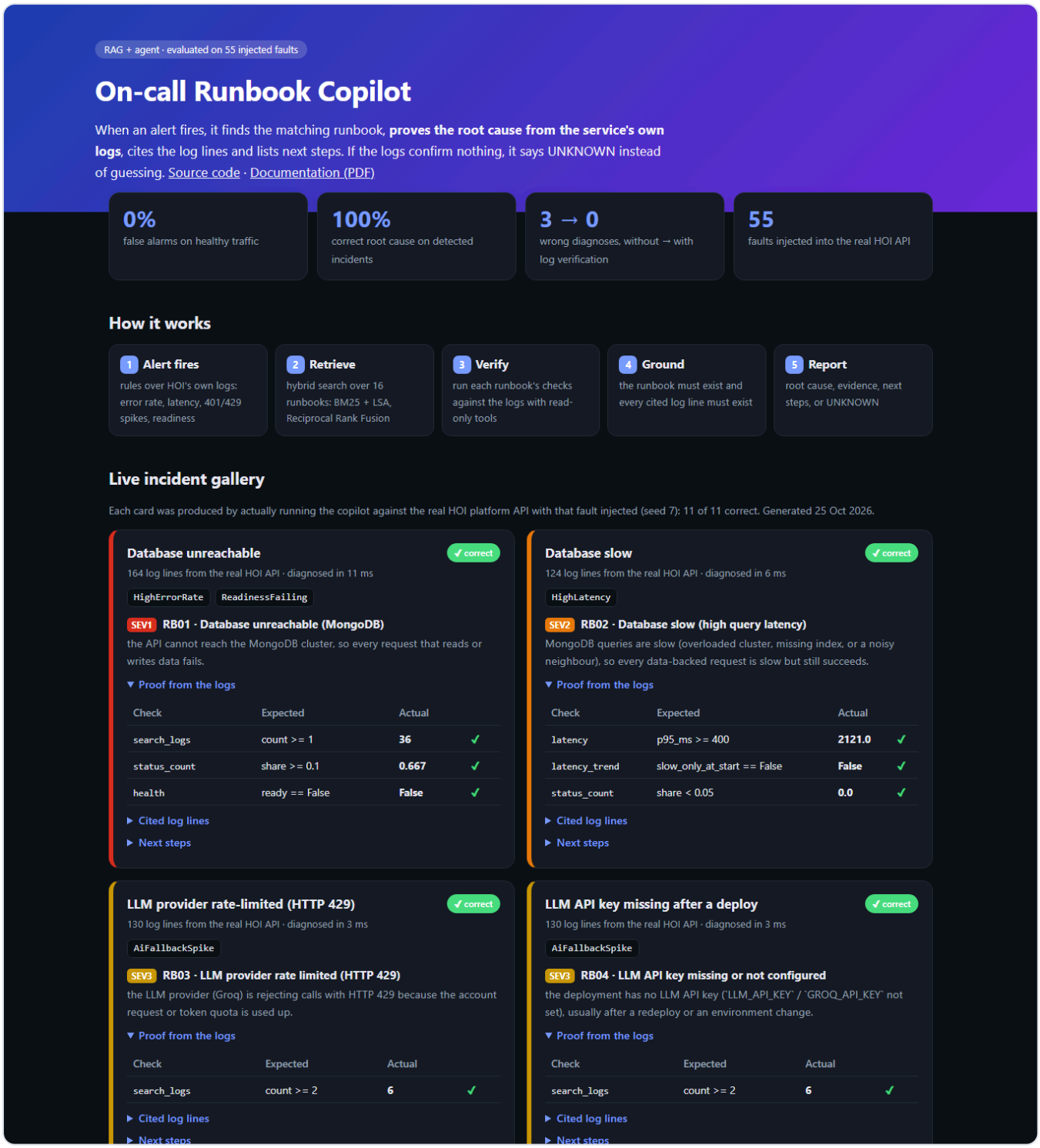
13. Interview questions

1 Overview

The On-call Runbook Copilot helps the engineer on call during an incident. When an alert fires on a production service, it finds the runbook that matches the symptoms, **checks that runbook's claims against the service's own logs**, and writes an incident note: the root cause, the log lines that prove it, and the next steps. If no runbook is confirmed by the logs, it answers **UNKNOWN** and escalates instead of guessing.

0% false alarms on healthy traffic	**100%** correct root cause on detected incidents	**3 → 0** wrong diagnoses, without → with log verification	**55** faults injected into a real API

It runs against a real system: the platform API of **HOI (Hospitality Operations Intelligence)**, a multi-tenant FastAPI + MongoDB service. HOI's code is not modified; faults are injected from outside by swapping dependencies HOI already isolates, so the copilot reads exactly the log lines HOI writes in production.



2 The problem

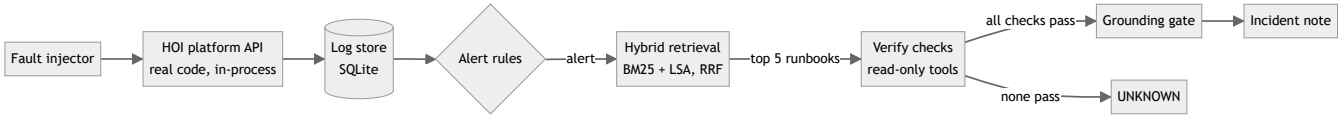
At 3 a.m. an alert says "38% of requests are failing". The engineer must work out *why*: is the database down, did a deploy break configuration, is someone attacking the login endpoint? Teams write **runbooks** ("if you see X, check A, B and C; the cause is usually D"), but under pressure people pick the wrong runbook, or trust a symptom that several different causes share.

The hard cases are **look-alike incidents**:

Symptom	Possible causes that look the same	What separates them
High latency	Slow database · cold start after idle	Is the slowness sustained, or only in the first requests?
Spike of HTTP 401	Signing secret rotated · password guessing · normal token expiry	Are the 401s spread over many endpoints, concentrated on `/auth/login`, or a trickle?
Spike of HTTP 429	Rate limit set too low · brute-force attack	Which endpoints get 429: guest chat across rooms, or the login endpoint?
AI answers degrade	Provider rate limit · missing API key · model output changed	The exact fallback reason in the logs

A pure search or LLM approach picks by *wording* and confuses these. The copilot picks by *evidence*.

3 Architecture



Component	File	Responsibility
Log store	`copilot/logstore.py`	Captures Python log records into SQLite; parses HOI's access lines (`rid=… GET /rooms -> 500 4ms`) into status, path and latency columns.
Tools	`copilot/tools.py`	Read-only queries: status counts, latency percentiles and trend, 401 spread, log search, top error templates, health probe.
Alert rules	`copilot/alerts.py`	Decide *what* looks wrong (error rate, readiness, latency, 401/429 spikes, AI fallbacks, dependency errors, startup failure).
Runbooks	`runbooks/*.md` , `copilot/runbooks.py`	16 Markdown runbooks with a machine-checkable `checks` block; split into section chunks for retrieval.
Retrieval	`copilot/retrieval.py`	BM25 written out by hand, LSA vectors (TF-IDF + truncated SVD), Reciprocal Rank Fusion.
Investigator	`copilot/agent.py`	Builds the query, verifies candidate runbooks, applies the grounding gate, renders the note.
LLM investigator	`copilot/llm_agent.py`	Optional: an LLM chooses the tools via function calling; same grounding gate.
Harness	`harness/`	Runs HOI in-process on an in-memory database, generates traffic, injects faults.
Evaluation	`eval/run_eval.py`	55 scenarios, five modes (ablation), metrics and misses.
Demo site	`site/build_site.py`	Runs every fault and publishes the incident gallery to GitHub Pages.

4 How an incident is investigated

1. Alert

Alert rules run over the log store. Each alert carries a summary and the ids of the log rows that triggered it. Thresholds are assumptions for a small service (tens of requests per window), for example *5xx share ≥ 10%* or *p95 latency ≥ 400 ms*.

2. Query

The investigator writes the query an engineer would type: the alert names and summaries, the three most frequent warning/error *templates* (log lines with ids, tokens and numbers replaced, so identical messages group together), and the most common AI fallback reason.

3. Retrieve

The query is ranked against runbook sections by two methods, which are fused by rank:

- **BM25**: exact words such as `ServerSelectionTimeoutError` or `429`. The tokenizer also splits CamelCase error names into words.
- **LSA vectors**: TF-IDF reduced to 48 dimensions with truncated SVD, which captures words that occur together ("slow", "latency", "p95").

4. Verify

For each of the top 5 runbooks, in rank order, the investigator runs the runbook's checks. A check names a tool, its arguments, a field, an operator and a value:

```
{"tool": "status_count", "args": {"status": "5xx"}, "field": "share", "op": ">=", "value": 0.1}
```

The first runbook whose checks **all** pass is the diagnosis. Look-alike runbooks contain checks that rule each other out (for example, "slow database" requires `slow_only_at_start == false`; "cold start" requires it to be true).

5. Ground and report

Before anything is shown, a grounding gate checks that the runbook exists and that every cited log id exists in the store. The note lists the alerts, the diagnosis and severity, the root cause, up to six cited log lines and the runbook's diagnosis and mitigation steps.

5 Retrieval in detail

BM25

For a query term *t* and a section *d*:

```
score(t, d) = idf(t) · f · (k1 + 1) / (f + k1 · (1 − b + b · |d| / avgdl))
idf(t)      = ln(1 + (N − df + 0.5) / (df + 0.5))
```

f is how often *t* occurs in *d*. **k1 = 1.5** limits how much repeating a word helps; **b = 0.75** corrects for long sections; the **+1** inside the logarithm keeps very common words from getting a negative weight.

Reciprocal Rank Fusion

```
rrf(runbook) = 1 / (60 + rank_bm25) + 1 / (60 + rank_dense)
```

RRF uses only ranks, so the two score scales (BM25 sums and cosine similarities) never need to be calibrated against each other. A runbook's score is that of its best section.

6 The runbooks

Id	Runbook	Severity	Injected in evaluation?
RB01	Database unreachable	SEV1	yes
RB02	Database slow	SEV2	yes
RB03	LLM provider rate limited (429)	SEV3	yes
RB04	LLM API key missing	SEV3	yes
RB05	LLM output no longer matches schema	SEV3	yes
RB06	Token signing secret changed	SEV2	yes
RB07	Password guessing on login	SEV2	yes
RB08	Tourism portal dependency down	SEV3/4	as a no-impact case
RB09	Unsafe production configuration	SEV1	yes
RB10	Rate limit set too low	SEV2	yes
RB11–RB16	Cold start, token expiry, inventory compensation, dispatch conflicts, CORS, memory pressure	various	no: near-miss decoys

7 Fault injection

Each fault swaps one dependency HOI already isolates and is undone afterwards, so scenarios cannot leak into each other. The investigation runs **while the fault is still active**, so health probes see the broken system as on a real on-call.

Fault	How it is injected	What HOI logs
Database down	MongoDB accessors raise `ServerSelectionTimeoutError`	500s with that exception; readiness failures
Database slow	A proxy sleeps before every collection call	Latency in seconds, all 200s
LLM rate limited	Chat adapter raises "HTTP 429"	`concierge fallback to rules: llm call failed (HTTP 429)`
LLM key missing	Keys removed from the environment	`… set LLM_API_KEY or GROQ_API_KEY`
Schema drift	Model returns prose or another JSON shape	`… invalid JSON for the schema`
Secret rotated	New `HOI_JWT_SECRET` after tokens were issued	401s across many endpoints
Password guessing	Many failed logins from one IP	401 then 429 on `/auth/login`
Rate limit too low	Guest chat limit set to 1–8 per minute	429 on guest chat
Unsafe prod config	Production mode without secrets	Startup failure (HOI refuses to boot)
Portal down	Places API times out or returns 503	Nothing: HOI serves cached places (correct answer: no incident)

8 Evaluation

55 scenarios: 11 types × 5 variants with different seeds, fault strengths and background noise (typo logins, 404s). Five modes form an ablation, to show what each part contributes.

Mode	Correct (all 55)	Correct (41 detected)	Wrong root cause
Hybrid retrieval + log-verified checks (the copilot)	93%	100%	0
Hybrid retrieval, top-1, no verification	87%	93%	3
BM25 only, top-1	87%	93%	3
LSA vectors only, top-1	87%	93%	3
BM25 + verification	93%	100%	0

- **Verification removes the wrong answers** (3 → 0): retrieval by wording confuses the look-alike pairs.
- **Dense retrieval adds nothing here**: 16 short runbooks full of exact error names favour keywords. It should matter with paraphrased alerts or many more runbooks.
- **All 4 misses are detection misses**, not wrong diagnoses: 8 and 12 password guesses stay under the login limit, and chat limits of 5 or 8 per minute were not hit by the test traffic.
- **No false alarms** on 10 scenarios with nothing wrong, including the portal outage that HOI absorbs.

> **Honest caveat.** The same person wrote the faults, the alert rules and the runbook checks, so these numbers are optimistic. They show the pipeline works end to end on a real codebase and how its parts compare; they are not an accuracy estimate for unseen production incidents.

9 LLM investigator (optional)

`copilot/llm_agent.py` lets an LLM choose the tools through function calling (any OpenAI-compatible API: Groq, OpenAI or a local Ollama). Its final answer must be JSON with a runbook id and cited log ids, and it passes the same grounding gate plus one more rule: **cited log ids must have been returned by a tool during this investigation**, so the model cannot invent evidence. It is implemented but not yet evaluated, so no accuracy is claimed for it.

10 Safety and design decisions

- **Read-only tools.** The copilot never restarts, rolls back or changes anything; a human applies the fix.
- **Abstain rather than guess.** A wrong root cause at 3 a.m. sends people down the wrong path; UNKNOWN escalates.
- **Evidence over wording.** Retrieval only proposes; checks against the logs decide.
- **No secrets in logs.** HOI never logs the database URI or guest messages; the copilot only reads what HOI writes.

11 Limitations

- Faults are single and clean; real incidents overlap (a slow database that then times out).
- About 60 requests per scenario from one process; thresholds are set for that scale.
- LSA is learned from 16 runbooks; a pretrained embedding model would generalise better and can be plugged in through `Embedder`.
- All HOI data is synthetic demo data.

12 Running it

```
..\INNOV8\.venv\Scripts\python.exe demo.py --list           # fault names
..\INNOV8\.venv\Scripts\python.exe demo.py db_down          # one incident note
..\INNOV8\.venv\Scripts\python.exe -m pytest -q tests       # 11 unit tests
..\INNOV8\.venv\Scripts\python.exe eval\run_eval.py         # 55 scenarios
..\INNOV8\.venv\Scripts\python.exe site\build_site.py       # rebuild the live gallery
$env:GROQ_API_KEY="..."; ..\INNOV8\.venv\Scripts\python.exe eval\run_eval.py --llm
```

13 Interview questions

Why not just ask an LLM what is wrong?

It would answer from wording and could invent causes. Here every diagnosis must be confirmed by checks on the real logs and cite log lines that exist; otherwise the answer is UNKNOWN.

What does BM25's k1 do? And b?

k1 caps the benefit of repeating a word (term-frequency saturation); b scales how much long documents are penalised.

Why Reciprocal Rank Fusion instead of adding scores?

BM25 scores and cosine similarities are on different scales; RRF only uses ranks, so no calibration is needed.

How do you know it works?

55 injected faults on a real API, five-mode ablation, zero false alarms, and verification taking wrong diagnoses from 3 to 0. And I state that the numbers are optimistic because I wrote the faults and the checks.

What would you do next?

Evaluate the LLM mode, add overlapping faults, use a pretrained embedding model, and test on runbooks and incidents written by someone else.